

HDLC Raw Programming Guide

Bypassing XMSG: Direct HDLC Communication on SINTRAN III

This guide covers how to send and receive raw data over HDLC without using XMSG (MON 200B). All communication uses **MON 201B (HDLCfunction)** to exchange Driver Control Blocks (DCBs) directly with the HDLC device driver. This is the foundation for building custom protocols over HDLC on ND-100 systems.

Audience: Programmers who need point-to-point data transfer over HDLC and want full control over the frame payload.

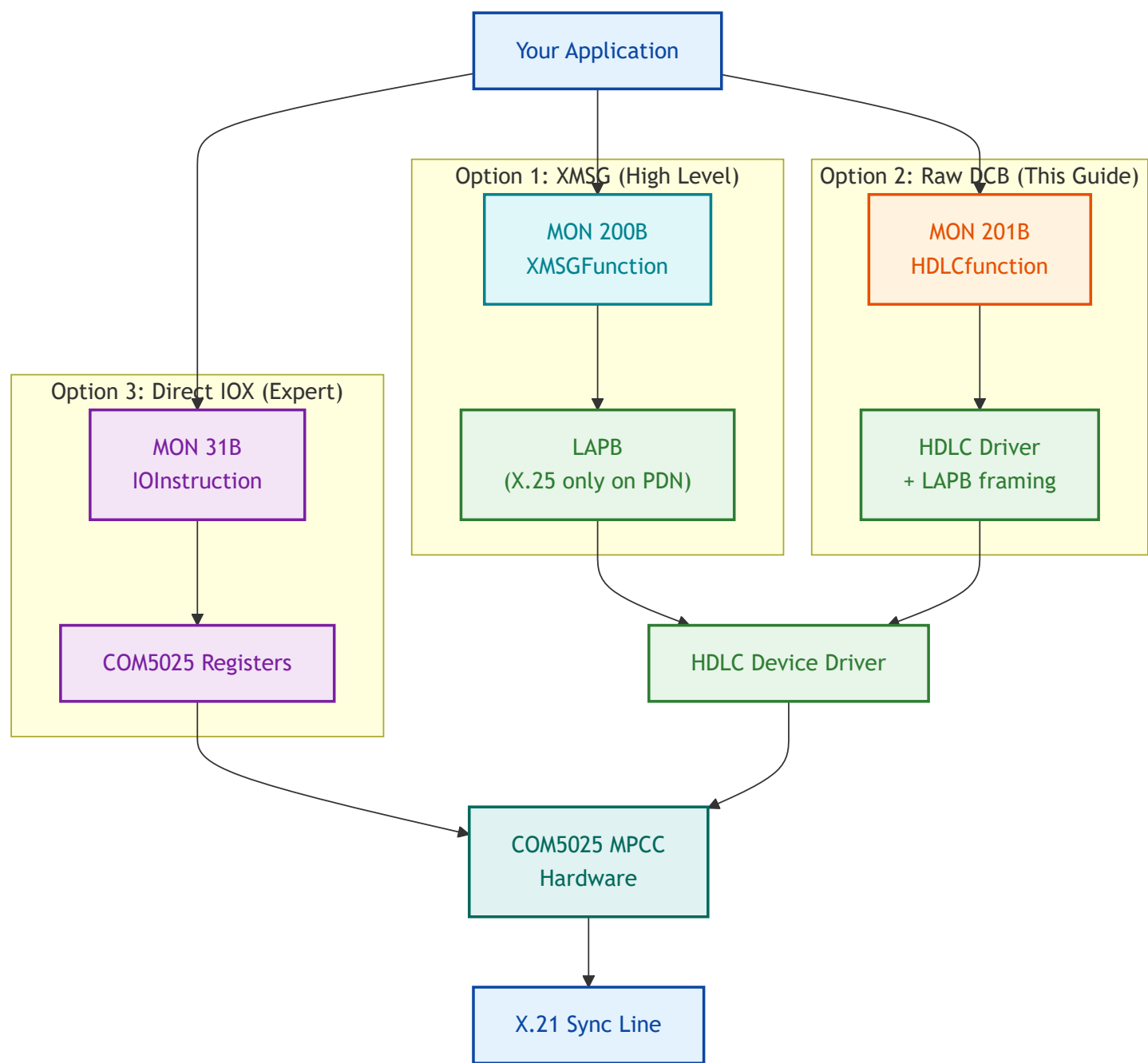
Primary Source: SINTRAN III Monitor Calls (ND-860228.2 EN), pages 306-307

Table of Contents

1. [Architecture Overview](#)
2. [Protocol Stack Comparison](#)
3. [Monitor Calls Reference](#)
4. [Driver Control Block \(DCB\) Structure](#)
5. [HDLC Frame Encapsulation](#)
6. [Programming Flow](#)
7. [Error Handling](#)
8. [Hardware Layer Reference](#)
9. [DMA Descriptor Format](#)
10. [Complete PLANC Source Code](#)
11. [Debugging Checklist](#)
12. [Appendix A: LDN Discovery](#)
13. [Appendix B: Advanced -- Direct Register Access](#)

1. Architecture Overview

There are three ways to communicate over HDLC on SINTRAN III, listed from highest to lowest level of abstraction:



Approach	MON Call	What You Control	What the System Handles
XMSG	MON 200B	Application messages	XMSG routing, LAPB, HDLC framing, CRC, DMA (X.25 only on PDN links)

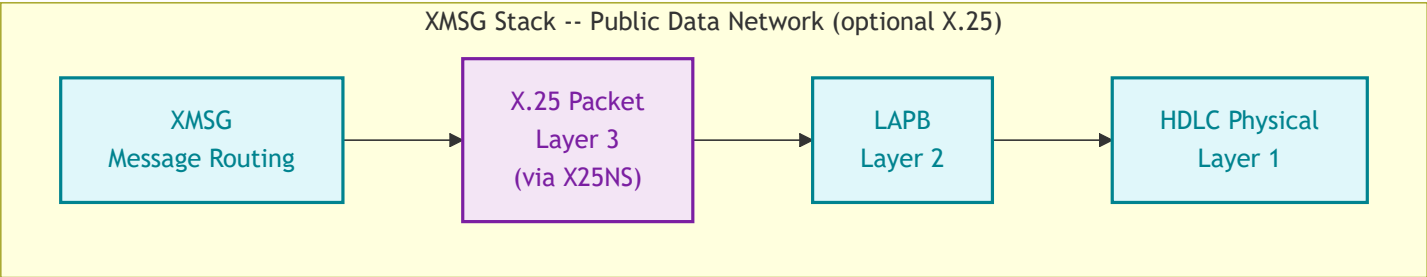
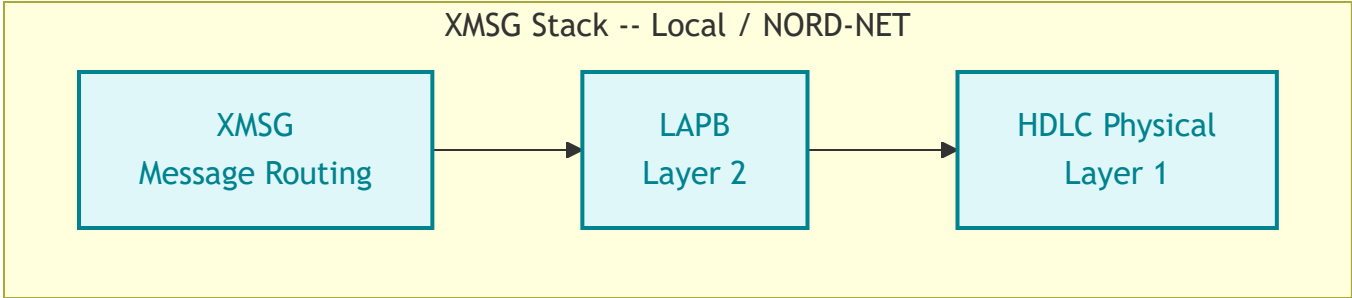
Approach	MON Call	What You Control	What the System Handles
Raw DCB	MON 201B	Frame payload (I-field content)	LAPB framing, HDLC flags, CRC, bit stuffing, DMA
Direct IOX	MON 31B	Everything: registers, DMA, timing	Nothing -- you are the driver

This guide focuses on **Option 2: Raw DCB via MON 201B**.

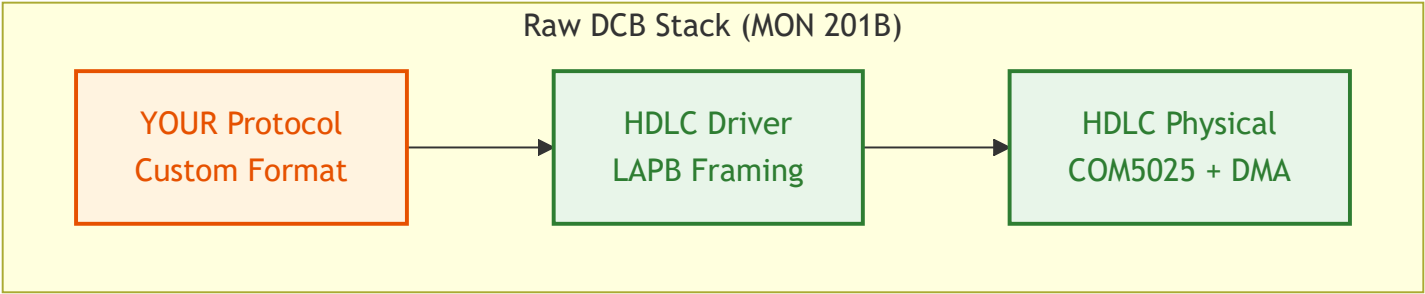
2. Protocol Stack Comparison

With XMSG (what you are bypassing)

XMSG can operate with or without X.25. On local NORD-NET links, XMSG sits directly on LAPB with no X.25 layer. X.25 is only inserted when connecting over public data networks (PDN) via the X.25 Network Server (X25NS).



With Raw DCB (what this guide teaches)



What each layer provides

Layer	XMSG Provides	Raw DCB Provides	You Must Provide
Framing (0x7E flags)	Automatic	Automatic	--
Bit stuffing	Automatic	Automatic	--
CRC/FCS	Automatic	Automatic	--
LAPB address/control	Automatic	Automatic	--
X.25 packet header	Only on PDN links (via X25NS)	NOT included	Only if connecting to public data network
Message routing	Automatic	NOT included	Your protocol
Flow control	XMSG handles	NOT included	Your protocol
Sequencing	LAPB + XMSG	LAPB only	Your protocol (if needed above LAPB)
Error recovery	LAPB retransmit	LAPB retransmit	--

3. Monitor Calls Reference

All monitor calls used for raw HDLC programming are documented below with their complete parameter specifications. Every call includes the PLANC calling convention.

3.1 MON 201B -- HDLCfunction (MHDLC)

Purpose: Send and receive raw HDLC frames via Driver Control Blocks. This is the primary call for all raw HDLC communication.

Source: SINTRAN III Monitor Calls (ND-860228.2 EN), page 306

Parameter	Name	Type	I/O	Description
1	Func	INTEGER	I	Function code: 0 = send DCB to driver, 1 = receive DCB from driver
2	DevNo	INTEGER	I	Logical Device Number. Separate LDNs for input and output parts
3	Buffer	ARRAY	I/O	Address of the Driver Control Block buffer
4	USize	INTEGER	I/O	Size of the used part of the DCB in bytes. On receive, updated to actual size
5	MSize	INTEGER	I	Send: maximum DCB buffer size in bytes. Receive: wait flag (1 = block until data, 0 = return immediately)
6	Status	INTEGER	O	Error code (see Error Codes)

PLANC calling convention:

```
INTEGER : Func, DevNo, USize, MSize, Status
INTEGER ARRAY : Buffer(0:1023)
```

```
Monitor_Call('HDLCfunction', Func, DevNo, Buffer, USize, MSize, Status)
```

Compatibility: ND-100 and ND-500 | All users | All programs

HDLCfunction Error Codes

Status Code	Meaning	Common Cause
0	Success	--
1	LDN not reserved	Forgot to call MON 122 (ReserveResource) first
2	Illegal LDN	Wrong device number; check with @STATUS-DEVICE
3	No DCB in queue	Receive with poll mode (MSize=0) and no data waiting
4	No buffer available	Driver out of internal buffers
5	Illegal DCB size	USize is zero, negative, or exceeds hardware limit
6	Illegal LDN for this call	Using input LDN for send or output LDN for receive
7	Max size < used size	MSize parameter smaller than USize
10 (8 decimal)	Illegal function code	Func is not 0 or 1
11 (9 decimal)	Fatal error	Hardware failure or driver crash

3.2 MON 122B -- ReserveResource (RESRV)

Purpose: Reserve exclusive access to an HDLC device. Must be called before any HDLCfunction calls. HDLC devices have separate input and output parts that must be reserved independently.

Source: SINTRAN III Monitor Calls (ND-860228.2 EN), page 419

Parameter	Name	Type	I/O	Description
1	DeviceNo	INTEGER	I	Logical device number (see Appendix B of Monitor Calls manual)
2	IOFlag	INTEGER	I	0 = reserve input part, 1 = reserve output part

Parameter	Name	Type	I/O	Description
3	WaitFlag	INTEGER	I	0 = wait if already reserved by another program, 1 = return status immediately
4	Status	INTEGER	O	Return status. Negative = already reserved by another program (only when WaitFlag=1)

PLANC calling convention:

INTEGER : DeviceNo, IOFlag, WaitFlag, Status

Monitor_Call('ReserveResource', DeviceNo, IOFlag, WaitFlag, Status)

Important: You must call this twice for HDLC -- once for the input LDN (IOFlag=0) and once for the output LDN (IOFlag=1). These are typically different logical device numbers.

Compatibility: ND-100 and ND-500 | All users | All programs

3.3 MON 123B -- ReleaseResource (RELES)

Purpose: Release a previously reserved device so other programs can use it. Must be called during cleanup for both input and output parts.

Source: SINTRAN III Monitor Calls (ND-860228.2 EN), page 411

Parameter	Name	Type	I/O	Description
1	DeviceNumber	INTEGER	I	Logical device number
2	IOFlag	INTEGER	I	0 = release input part, 1 = release output part

PLANC calling convention:

INTEGER : DeviceNumber, IOFlag

Monitor_Call('ReleaseResource', DeviceNumber, IOFlag)

Note: A normal termination of an RT program releases all resources automatically. However, explicit release is good practice and required if the program continues running after HDLC communication ends.

Compatibility: ND-100 and ND-500 | All users | All programs

3.4 MON 141B -- DeviceControl (IOSET)

Purpose: Set control mode for a character device. Used to switch the HDLC device between ASCII and binary mode, or to reset it. The device must be reserved first.

Source: SINTRAN III Monitor Calls (ND-860228.2 EN), page 141

Parameter	Name	Type	I/O	Description
1	DeviceNo	INTEGER	I	Logical device number (cannot use 1 for own terminal; use ExecutionInfo)
2	IOFlag	INTEGER	I	0 = input part, 1 = output part
3	RTProgram	INTEGER	I	Address of RT description of reserving program. Use 0 for calling program
4	CtrlFlag	INTEGER	I	Control flag (see table below)
5	ReturnStatus	INTEGER	O	0 = success, -1 = illegal RT description address

CtrlFlag values:

Value	Mode	Description
-2	Empty TAD buffer	Flush the TAD output buffer
-1	Reset device	Reset to ASCII mode
0	ASCII mode	Standard character-oriented I/O
1	Binary mode	Raw binary data transfer (use this for HDLC)

PLANC calling convention:

INTEGER : DeviceNo, IOFlag, RTProgram, CtrlFlag, ReturnStatus

```
Monitor_Call('DeviceControl', DeviceNo, IOFlag, RTProgram, CtrlFlag, ReturnStatus)
```

Compatibility: ND-100 and ND-500 | All users | All programs

3.5 MON 31B -- IOInstruction (EXIOX)

Purpose: Execute a raw IOX machine instruction to read or write hardware device registers directly. This is the lowest-level access to the COM5025 HDLC controller. Requires that the register address be pre-registered in the IOX table via SINTRAN-SERVICE-PROGRAM command `INSERT-IN-IOX-TABLE` .

Source: SINTRAN III Monitor Calls (ND-860228.2 EN), page 327

Parameter	Name	Type	I/O	Description
1	RegContents	INTEGER	I	Value to write to the device register
2	DevRegAddr	INTEGER	I	Device register address (IOX port number)
3	ContentsAfter	INTEGER	O	Register contents after execution

PLANC calling convention:

INTEGER : RegContents, DevRegAddr, ContentsAfter

```
Monitor_Call('IOInstruction', RegContents, DevRegAddr, ContentsAfter)
```

Warning: This call bypasses the HDLC driver entirely. Incorrect register writes can crash the HDLC controller, corrupt DMA, or hang the system. Only use for diagnostics or when building a custom driver.

Compatibility: ND-100 and ND-500 | User RT and user SYSTEM | All programs

3.6 MON 143B -- ExecutionInfo (RSIO)

Purpose: Get the logical device number of the calling program's terminal.

Required because MON 162B (OutString) cannot use device number 1 -- you must pass the actual terminal LDN.

Source: SINTRAN III Monitor Calls (ND-860228.2 EN)

Parameter	Name	Type	I/O	Description
1	ExecutionMode	INTEGER	O	0 = interactive, 1 = batch, 2 = mode job, 3 = RT program
2	InputDev	INTEGER	O	LDN for command input (terminal number for interactive programs)
3	OutputDev	INTEGER	O	LDN for command output (terminal number for interactive programs)
4	UserIndex	INTEGER	O	Directory index (bits 8-15) and user index (bits 0-7)

PLANC calling convention:

INTEGER : ExecMode, InDev, OutDev, UserIdx

```
Monitor_Call('ExecutionInfo', ExecMode, InDev, OutDev, UserIdx)
```

Compatibility: ND-100 and ND-500 | All users | All programs

3.7 Output Monitor Calls (for diagnostics and printing)

These calls are used in the example code for printing status messages and buffer dumps to the terminal.

MON 32B -- OutMessage (MSG)

Writes a string to the user's terminal. Maximum 512 characters.

Parameter	Name	Type	I/O	Description
1	Message	BYTES	I	String to write (max 512 characters)

```

BYTES : Message(0:79)
Monitor_Call('OutMessage', Message)

```

Compatibility: ND-100 and ND-500 | All users | Background programs

MON 35B -- OutNumber (IOUT)

Writes a number to the user's terminal in the specified radix.

Parameter	Name	Type	I/O	Description
1	Format	INTEGER	I	Output radix: 2 = binary, 8 = octal, 10 = decimal, 16 = hex
2	Number	INTEGER	I	Value to print (-32768 to 32767)

```

INTEGER : Format, Number
Monitor_Call('OutNumber', Format, Number)

```

Compatibility: ND-100 and ND-500 | All users | Background programs

MON 2B -- OutByte (OUTBT)

Writes one byte to a character device.

Parameter	Name	Type	I/O	Description
1	DeviceNumber	INTEGER	I	Logical device number (use 1 for own terminal)
2	OutputValue	INTEGER	I	Byte value to write

```

INTEGER : DeviceNumber, OutputValue
Monitor_Call('OutByte', DeviceNumber, OutputValue)

```

Compatibility: ND-100 and ND-500 | All users | All programs

MON 162B -- OutString (OUTST)

Writes a string to a peripheral device. Cannot use device number 1 for own terminal -- use ExecutionInfo to get the actual LDN first.

Parameter	Name	Type	I/O	Description
1	DeviceNo	INTEGER	I	Logical device number (NOT 1 ; use ExecutionInfo)
2	TextWrite	BYTES	I	Character string to output (max 2048 bytes)
3	NoOfBytes	INTEGER	I	Number of bytes to write
4	ReturnStatus	INTEGER	O	Error status

INTEGER : DeviceNo, NoOfBytes, ReturnStatus

BYTES : TextWrite(0:79)

Monitor_Call('OutString', DeviceNo, TextWrite, NoOfBytes, ReturnStatus)

Compatibility: ND-100 and ND-500 | All users | All programs

3.8 Complete MON Call Summary Table

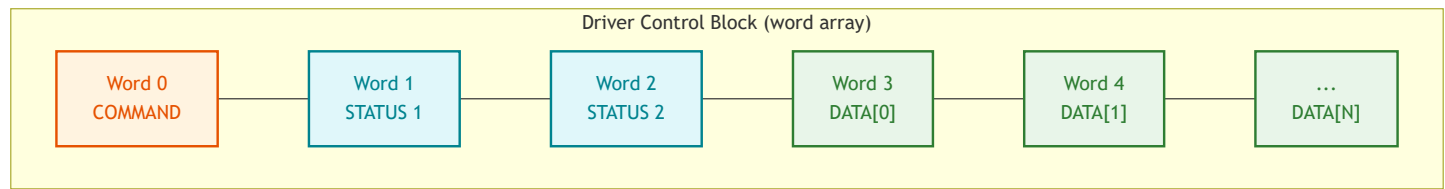
MON Octal	Name	Short	Purpose	Required for Raw HDLC?
122B	ReserveResource	RESRV	Reserve device for exclusive use	Yes -- must reserve both input and output LDNs
123B	ReleaseResource	RELES	Release reserved device	Yes -- cleanup
141B	DeviceControl	IOSET	Set device mode (ASCII/binary/reset)	Optional -- set binary mode if needed
143B	ExecutionInfo	RSIO	Get terminal LDN	Optional -- for diagnostic output via OutString

MON Octal	Name	Short	Purpose	Required for Raw HDLC?
201B	HDLCfunction	MHDLC	Send/receive DCBs to HDLC driver	Yes -- core communication call
31B	IOInstruction	EXIOX	Direct hardware register access	No -- expert/diagnostic use only
32B	OutMessage	MSG	Print string to terminal	Optional -- diagnostics
35B	OutNumber	IOUT	Print number to terminal	Optional -- diagnostics
2B	OutByte	OUTBT	Print single byte	Optional -- diagnostics
162B	OutString	OUTST	Print string to device	Optional -- diagnostics

4. Driver Control Block (DCB) Structure

The DCB is a word array passed to MON 201B. It is the fundamental data structure for all HDLC communication. The first three words are a fixed header; the remainder carries your payload.

4.1 DCB Memory Layout



4.2 DCB Word Definitions

Word Offset	Field	I/O	Size	Description
0	Command	I	1 word (16 bits)	DCB command code (see command table below)

Word Offset	Field	I/O	Size	Description
1	Status1	O	1 word (16 bits)	Primary status returned by driver
2	Status2	O	1 word (16 bits)	Secondary status / hardware status
3 .. N	Data	I/O	Variable	Frame payload -- your protocol data goes here

4.3 DCB Commands

The command word (DCB word 0) tells the HDLC driver what operation to perform.

Command Code	Name	Description	Data Words Used?
1	TRANS	Transfer frame data. Sends or receives an HDLC I-frame payload	Yes -- words 3..N contain frame data
2	RESET	Reset the logical device. Resets the LAPB link layer	No
3	DEVCL	Device clear. Clears device state and buffers	No
4	DEVINI	Device initialization. Must be called before first TRANS	No
5	DEVSTA	Get device status. Returns hardware/link status in Status1/Status2	No -- status returned in words 1-2

4.4 DCB Size Calculation

The USize parameter to MON 201B is the total DCB size **in bytes**:

$$USize = (HeaderWords + DataWords) * 2$$

Where:

$$HeaderWords = 3 \quad (\text{command} + \text{status1} + \text{status2})$$

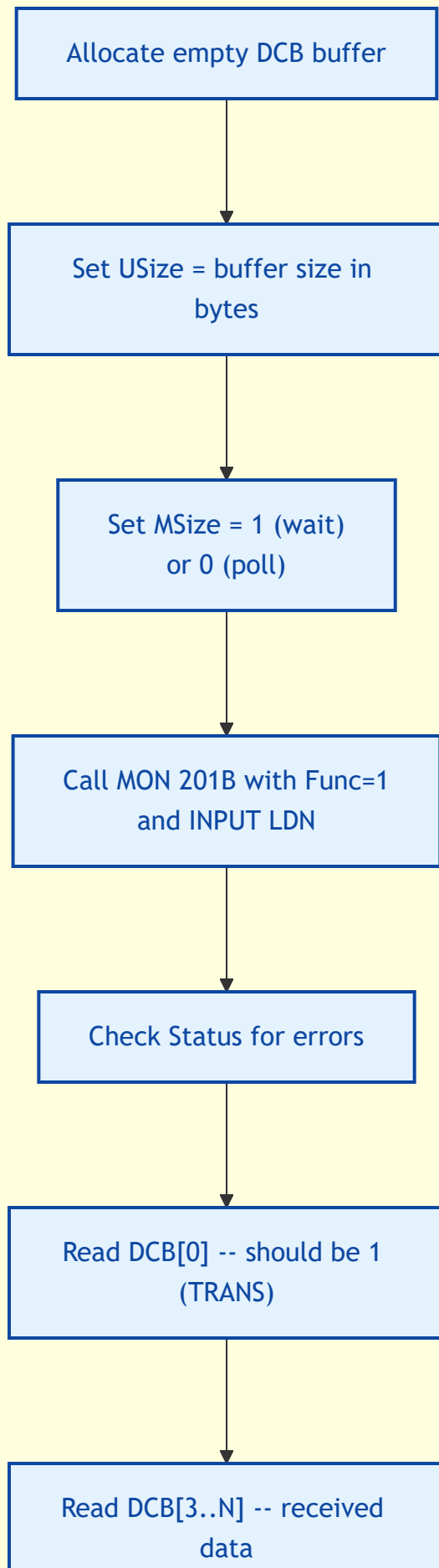
$$DataWords = \text{number of payload words}$$

Example: sending 8 words of data

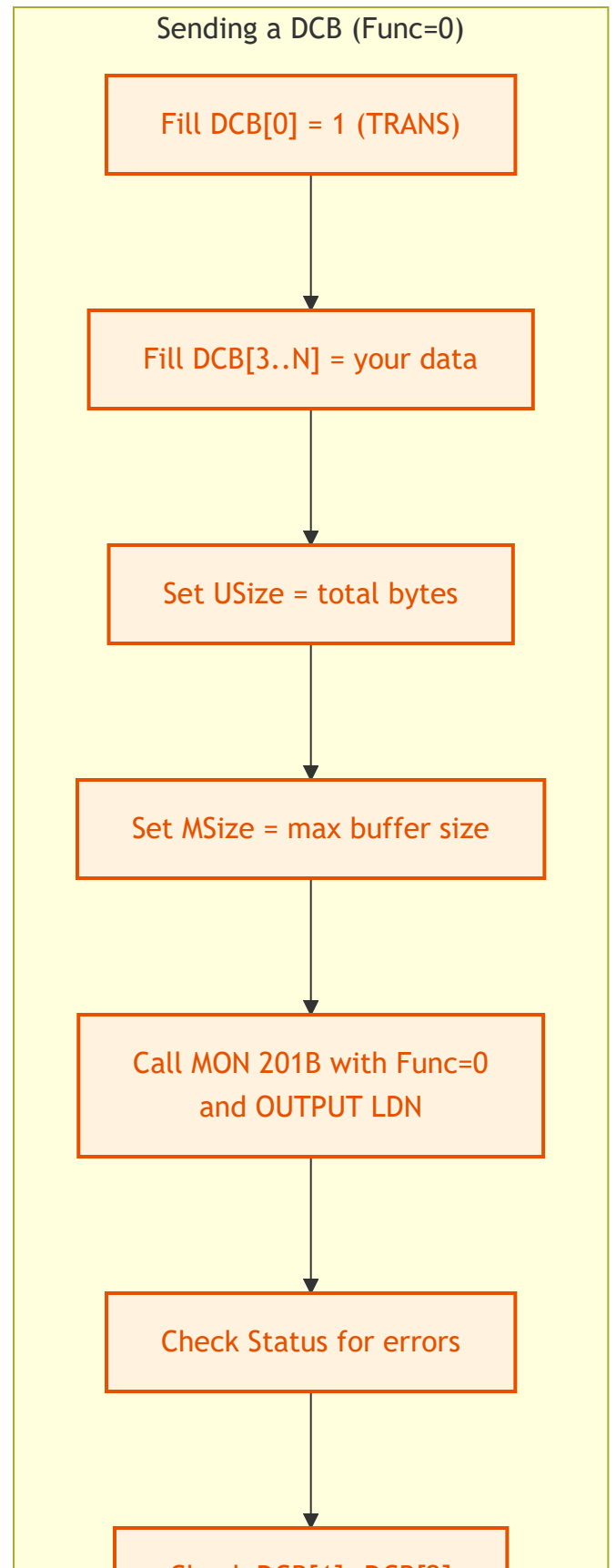
$$USize = (3 + 8) * 2 = 22 \text{ bytes}$$

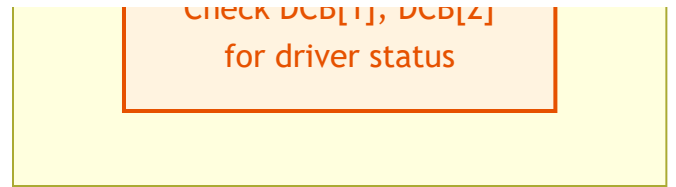
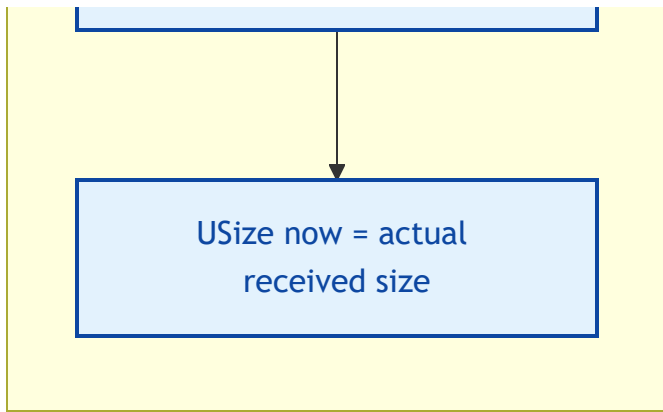
4.5 Send vs. Receive DCB Usage

Receiving a DCB (Func=1)



Sending a DCB (Func=0)

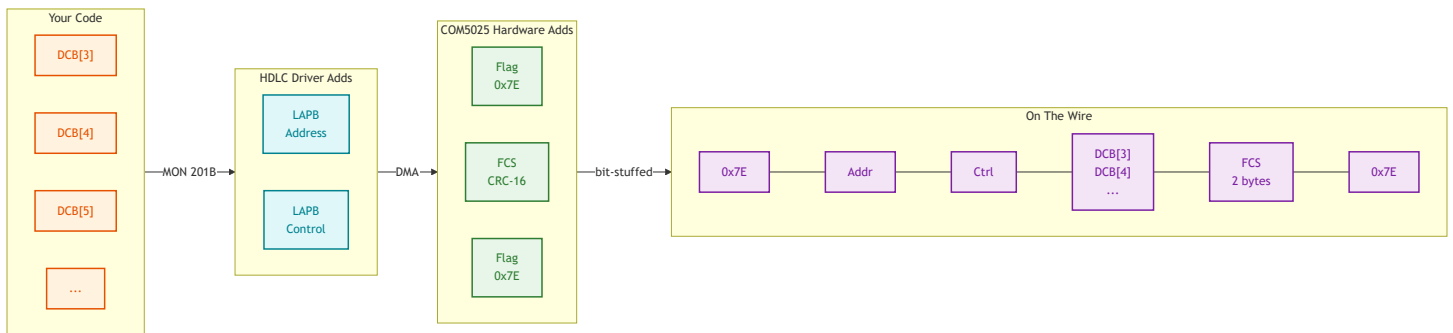




5. HDLC Frame Encapsulation

When you send a TRANS DCB, the driver and hardware wrap your data in a complete HDLC frame before putting it on the wire.

5.1 What happens to your DCB data



5.2 Wire Frame Structure

Byte Offset	Field	Size	Who Generates	Description
0	Opening Flag	1 byte	COM5025 hardware (TSOM bit)	0x7E (01111110)
1	LAPB Address	1 byte	HDLC driver	Single-byte LAPB addressing
2	LAPB Control	1 byte	HDLC driver	I/S/U frame type + N(S)/N(R) sequence

Byte Offset	Field	Size	Who Generates	Description
3 .. N+2	Information	N bytes	Your DCB data	DCB words 3..N, your protocol payload
N+3	FCS	2 bytes	COM5025 hardware	CRC-16-CCITT over Address + Control + Information
N+5	Closing Flag	1 byte	COM5025 hardware (TEOM bit)	0x7E

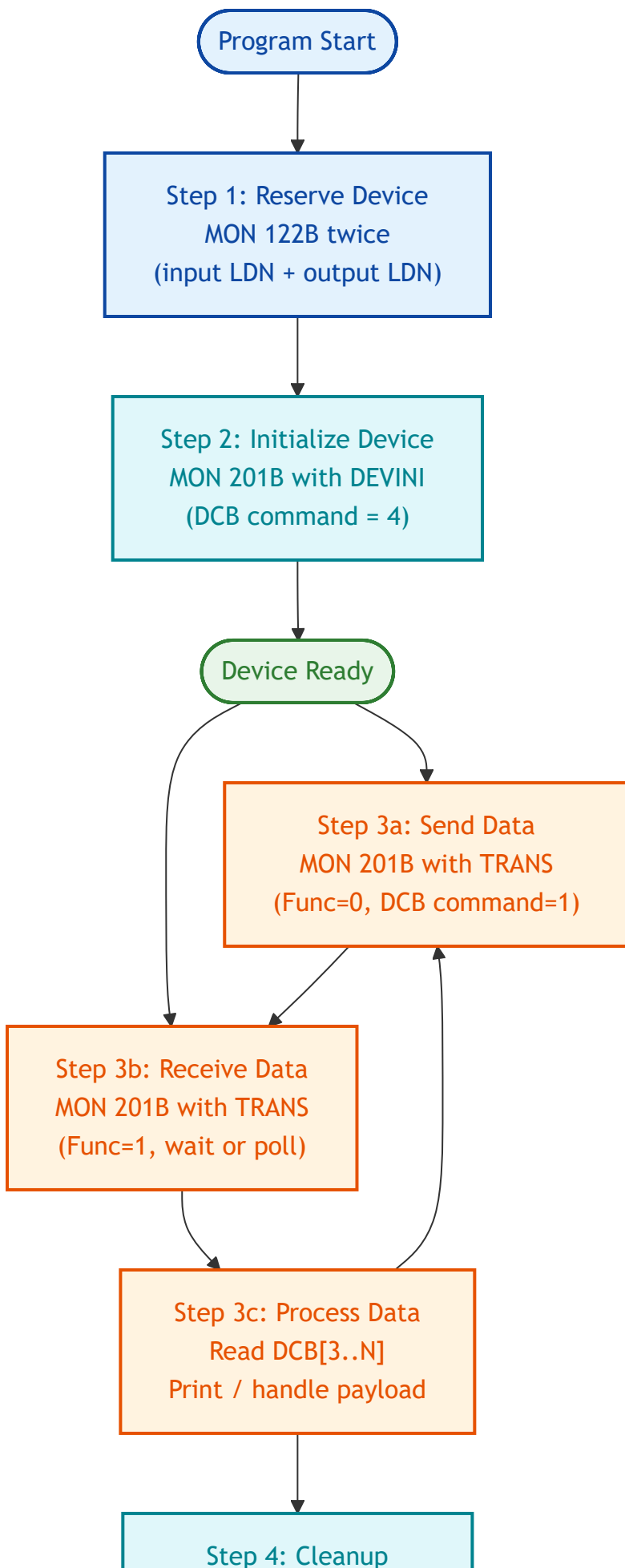
5.3 What you do NOT need to implement

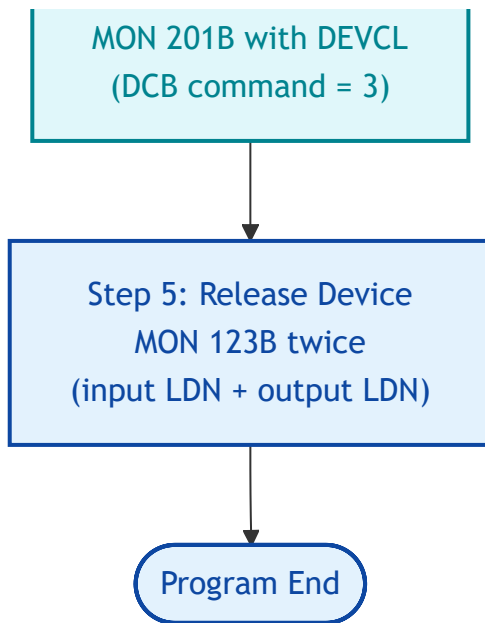
The HDLC driver and COM5025 hardware handle all of these automatically:

Feature	Handled By	Details
Flag generation (0x7E)	COM5025 TSOM/TEOM bits	Opening and closing flags
Bit stuffing	COM5025 hardware	Zero insertion after 5 consecutive 1-bits
CRC calculation	COM5025 hardware	CRC-16-CCITT (polynomial 0x1021)
CRC verification	COM5025 hardware	Bad CRC frames are discarded
LAPB framing	HDLC driver	Address and control byte management
LAPB sequencing	HDLC driver	N(S)/N(R) modulo-8 counters
LAPB retransmission	HDLC driver	Automatic retransmit on NAK or timeout
DMA transfer	HDLC driver	Buffer-to-hardware DMA descriptor setup

6. Programming Flow

6.1 Complete Session Lifecycle





6.2 Step-by-Step with DCB Contents

Step 1: Reserve the HDLC Device

Both the input and output logical device numbers must be reserved. They are typically different numbers (e.g., 560 for input, 561 for output).

```
% Reserve input part
0 =: IOFlag                                % 0 = input
0 =: WaitFlag                             % 0 = wait for it
Monitor_Call('ReserveResource', INLDN, IOFlag, WaitFlag, Status)

% Reserve output part
1 =: IOFlag                                % 1 = output
Monitor_Call('ReserveResource', OUTLDN, IOFlag, WaitFlag, Status)
```

Step 2: Initialize the HDLC Link

Send a DEVINI command DCB to bring the device to a known state.

```

4 =: DCB(0) % DEVINI command
0 =: DCB(1) % Clear status
0 =: DCB(2) % Clear status
0 =: Func % Send DCB to driver
6 =: USize % 3 words * 2 bytes
2048 =: MSize % Max buffer

```

```
Monitor_Call('HDLCfunction', Func, OUTLDN, DCB, USize, MSize, Status)
```

Step 3a: Send Data

Fill the DCB with a TRANS command and your payload starting at word 3.

```

1 =: DCB(0) % TRANS command
0 =: DCB(1) % Clear status
0 =: DCB(2) % Clear status
% DCB(3) .. DCB(N) = your payload
(PayloadWords + 3) * 2 =: USize % Total DCB size in bytes
0 =: Func % Send to driver

```

```
Monitor_Call('HDLCfunction', Func, OUTLDN, DCB, USize, MSize, Status)
```

Step 3b: Receive Data

Provide an empty buffer. The driver fills it with the received frame payload.

```

1 =: Func % Receive from driver
(MaxWords + 3) * 2 =: USize % Buffer capacity
1 =: MSize % 1 = block until data

```

```
Monitor_Call('HDLCfunction', Func, INLDN, DCB, USize, MSize, Status)
```

```

% On return:
% DCB(0) = command (should be 1 = TRANS for data frames)
% DCB(1) = driver status
% DCB(2) = hardware status
% DCB(3..N) = received payload
% USize = actual received size in bytes

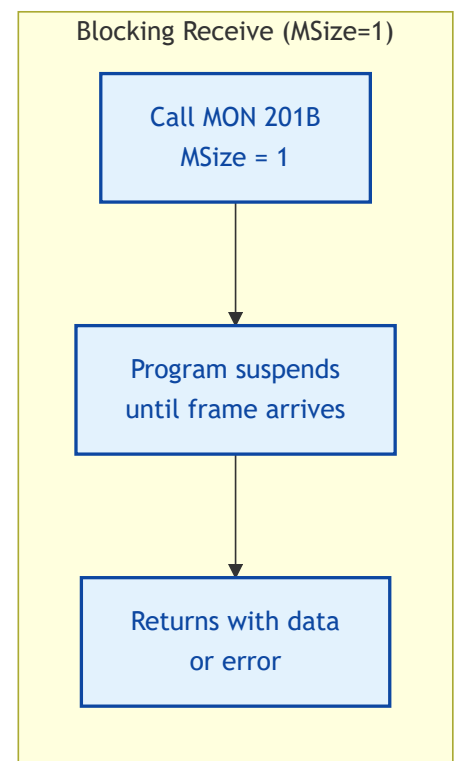
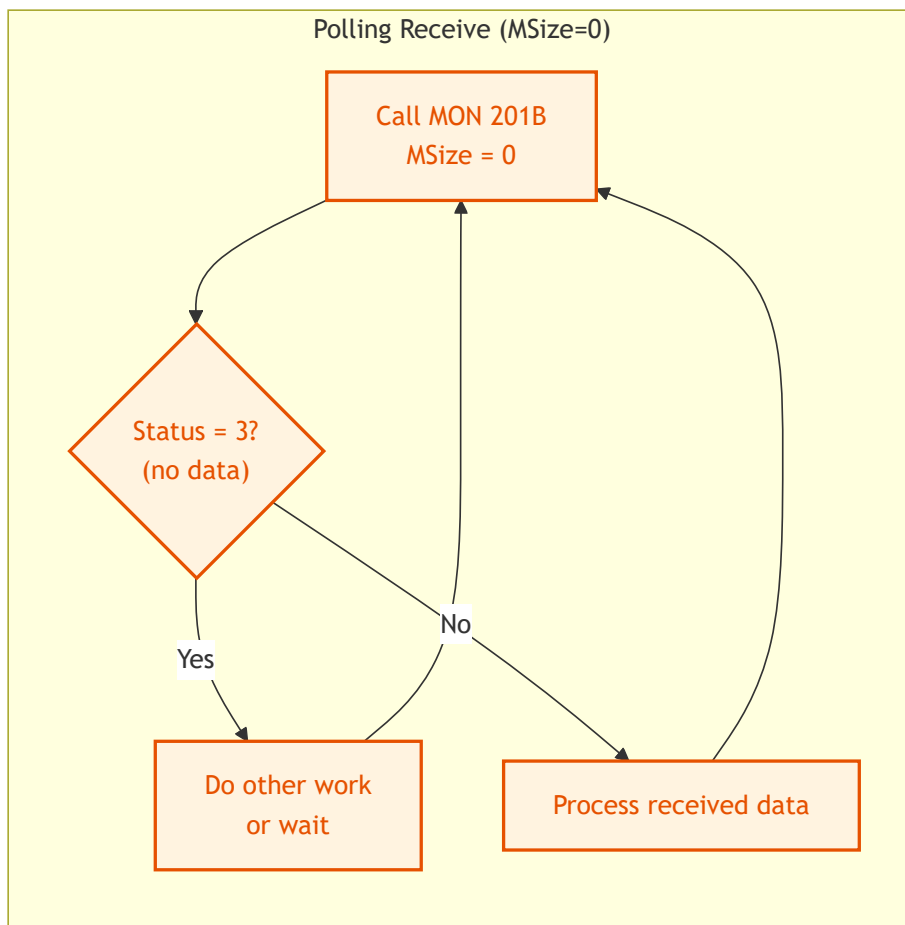
```

Step 4: Cleanup and Release

```
% Send DEVCL to clear the device
3 =: DCB(0)                                % DEVCL command
0 =: Func
6 =: USize
Monitor_Call('HDLCfunction', Func, OUTLDN, DCB, USize, MSize, Status)

% Release both parts
0 =: IOFlag
Monitor_Call('ReleaseResource', INLDN, IOFlag)
1 =: IOFlag
Monitor_Call('ReleaseResource', OUTLDN, IOFlag)
```

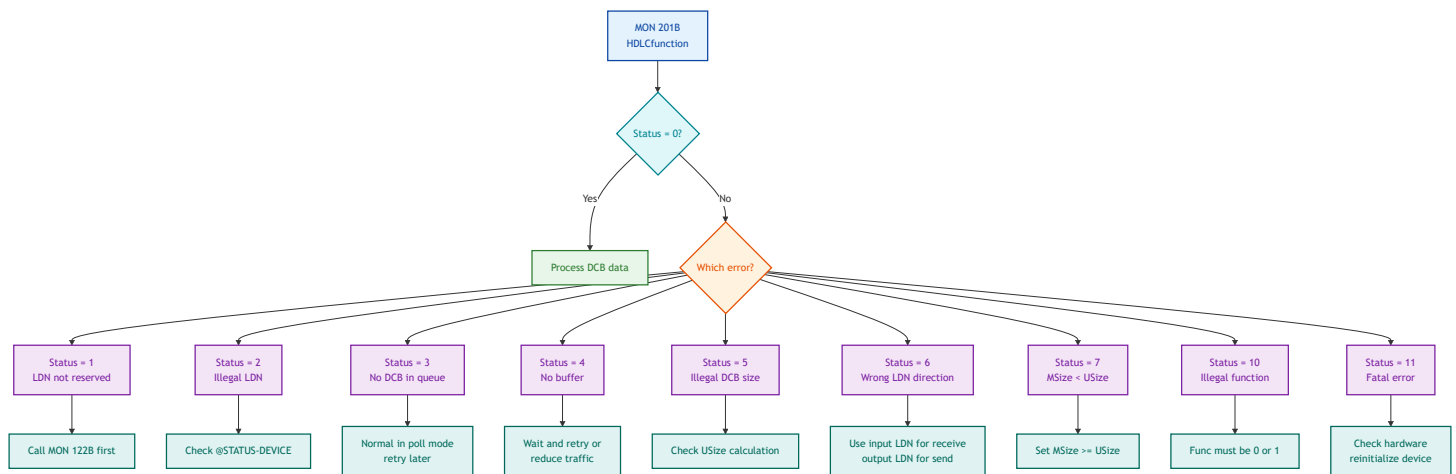
6.3 Blocking vs. Polling Receive



Mode	MSize Value	Behaviour	Use When
Blocking	1	Program waits until a frame arrives or an error occurs	Simple request-response protocols
Polling	0	Returns immediately with Status=3 if no data in queue	Programs that must do other work between checks

7. Error Handling

7.1 Error Handling Flow



7.2 DCB Status Words (returned by driver)

After a successful MON 201B call (Status = 0), check the DCB status words for hardware-level results:

DCB Word	Field	Bit(s)	Meaning
1	Status1	15 (SILFO)	Transmitter illegal format error
1	Status1	1 (TXUND)	Transmitter underrun (data not fed fast enough)
1	Status1	0	Data available (receiver)
1	Status1	11 (EMTY)	Receiver buffer list empty -- FATAL

DCB Word	Field	Bit(s)	Meaning
1	Status1	13-14	X.21 protocol errors
2	Status2	--	Additional hardware-specific status

Transmit success test: (Status1 AND 100002B) = 0 (both SILFO and TXUND clear)

Receive success test: (Status1 AND 1B) = 1 AND (Status1 AND 60000B) = 0 AND (Status1 AND 4000B) = 0

8. Hardware Layer Reference

This section documents the COM5025 registers accessible via MON 31B for advanced diagnostics or custom driver development.

8.1 COM5025 Register Map

All registers are at offsets from the HDLC device base address (HDEV).

IOX Offset	Register	R/W	Purpose
+10	RRTS	R	Receiver Transfer Status -- check after receive interrupt
+11	WRTC	W	Receiver Transfer Control -- enable/configure receiver
+12	RTTS	R	Transmitter Transfer Status -- check after transmit interrupt
+13	WTTC	W	Transmitter Transfer Control -- enable/configure transmitter
+15	WDMA	W	Write DMA list base address
+17	WDCR	W	Write DMA Command Register -- start DMA operations

8.2 Key Register Values

Register	Value (Octal)	Value (Hex)	Purpose
WRTC	1734	0x03DC	Full operational receiver mode (all interrupts enabled)

Register	Value (Octal)	Value (Hex)	Purpose
WRTC	100	0x0040	Minimal cleanup mode
WRTC	0	0x0000	Receiver off
WTTC	1134+CMODI	0x025C+	Full operational transmitter mode
WTTC	0	0x0000	Transmitter off
WDCR	1001	0x0201	Start receiver DMA
WDCR	2000	0x0400	Start transmitter DMA

8.3 RRTS Bit Map (Receiver Status)

Bit	Name	Normal Value	Meaning
0	DataAvailable	1	Packet ready for processing
11	ListEmpty (EMPTY)	0	FATAL if 1 -- no more receive buffers
13	X21D	0	X.21 data error
14	X21S	0	X.21 signal error

8.4 RTTS Bit Map (Transmitter Status)

Bit	Name	Normal Value	Meaning
1	TXUND (Underrun)	0	DMA underrun -- retry transmission
15	SILFO (Illegal)	0	Illegal format -- retry or abort

9. DMA Descriptor Format

The HDLC driver uses 4-word DMA descriptors to interface with the COM5025 hardware. Understanding this is useful for debugging and for the direct IOX approach.

9.1 Descriptor Layout

Word	Field	Description
0	LKEY	Block control (bits 10-8) + COM5025 command bits (bits 7-0)
1	LBYTC	Byte count -- number of data bytes in this block
2	LMEM1	Memory bank (high-order address bits)
3	LMEM2	Buffer address (low-order 16-bit address within bank)

9.2 LKEY Field Structure

Bits 15-11: Reserved / extended control

Bit 10: Legal Key flag (must be 1 for valid descriptor)

Bits 9-8: Block status

Bits 7-0: COM5025 register bits (TSOM, TEOM, TABORT, TGA)

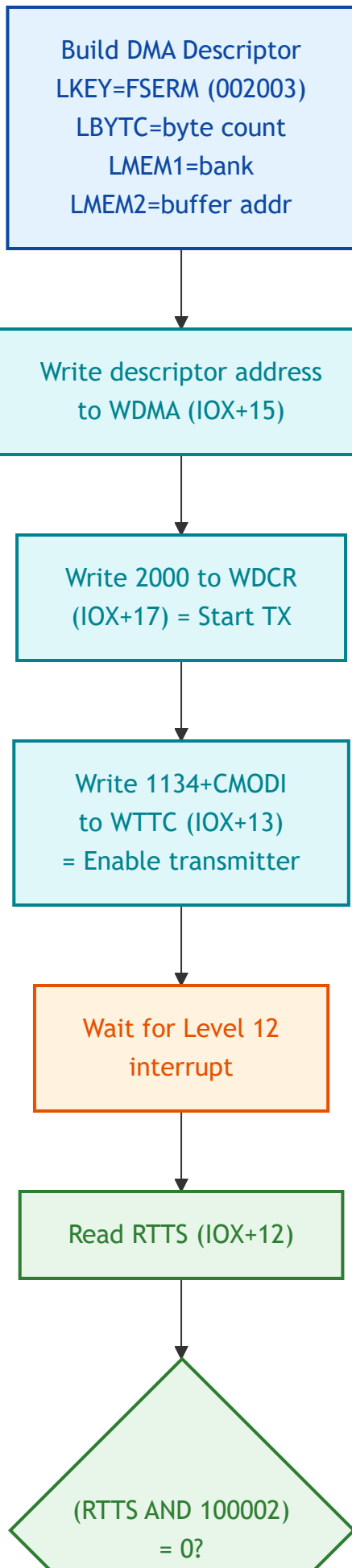
9.3 LKEY Values for Transmission

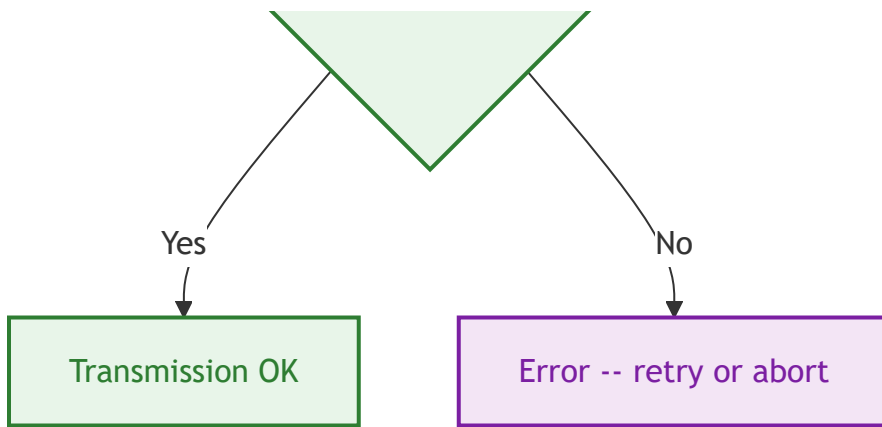
LKEY Value	Octal	Purpose	COM5025 Action
0x0403 (FSERM)	002003	Single complete frame	TSOM + TEOM: generate opening flag, data, CRC, closing flag
0x0401	002001	First block of multi-block frame	TSOM only: opening flag, start data
0x0400	002000	Middle block of multi-block frame	No flags: continue data
0x0402	002002	Last block of multi-block frame	TEOM only: finish data, CRC, closing flag

9.4 LKEY Values for Reception

LKEY Value	Octal	Status	Meaning
0x0400	002000	Empty Receiver Block	Available for incoming data
0x0600	003000	Full Receiver Block	Contains received data -- process it

9.5 DMA Transmit Sequence





10. Complete PLANC Source Code

A working PLANC program that sends a test buffer over HDLC and receives a response, printing both to the terminal.

```

%
% HDLC-RAW.PLANC -- Send and receive raw data over HDLC
% Uses MON 201B (HDLCfunction) directly, bypassing XMSG
%
% DCB layout:
%   Word 0: Command (1=TRANS, 2=RESET, 3=DEVCL, 4=DEVINI, 5=DEVSTA)
%   Word 1: Status returned by driver
%   Word 2: Additional status
%   Word 3+: Frame data payload
%

```

```

MODULE HDLCRAW

```

```

INTEGER ARRAY : STACK(0:2000)

```

```

%
% DCB buffers -- 1024 words max each
% Word 0 = command, Word 1-2 = status, Word 3+ = data
%
INTEGER ARRAY : TXDCB(0:1023)
INTEGER ARRAY : RXDCB(0:1023)

```

```

%
% Test payload to send
%
INTEGER ARRAY : TESTDATA(0:15)

```

```

%
% Receive buffer for incoming data
%
INTEGER ARRAY : RXDATA(0:255)

```

```

%-----
% Print a string to the user terminal
%-----
ROUTINE PRINTMSG(BYTES : MSG)
    Monitor_Call('OutMessage', MSG)
ENDROUTINE

```

```

%-----
% Print an integer in decimal

```



```
%-----  
ROUTINE PRINTNUM(INTEGER : VAL)  
    INTEGER : FMT  
    10 =: FMT  
    Monitor_Call('OutNumber', FMT, VAL)  
ENDROUTINE
```

```
%-----  
% Print an integer in octal  
%-----  
ROUTINE PRINTOCT(INTEGER : VAL)  
    INTEGER : FMT  
    8 =: FMT  
    Monitor_Call('OutNumber', FMT, VAL)  
ENDROUTINE
```

```
%-----  
% Print CR/LF to the terminal  
%-----  
ROUTINE NEWLINE  
    INTEGER : DEV, VAL  
    1 =: DEV  
    15B =: VAL  
    Monitor_Call('OutByte', DEV, VAL)  
    12B =: VAL  
    Monitor_Call('OutByte', DEV, VAL)  
ENDROUTINE
```

```
%-----  
% Reserve both input and output parts of HDLC device  
% Returns 0 on success, negative on failure  
%-----  
INTEGER ROUTINE RESERVE_HDLC(INTEGER : INLDN, OUTLDN)  
    INTEGER : IOFLAG, WAITFLAG, STATUS  
  
    % Reserve input part  
    0 =: IOFLAG  
    0 =: WAITFLAG  
    Monitor_Call('ReserveResource', INLDN, IOFLAG, WAITFLAG, STATUS)
```

```

% Reserve output part
1 =: IOFLAG
0 =: WAITFLAG
Monitor_Call('ReserveResource', OUTLDN, IOFLAG, WAITFLAG, STATUS)

RETURN (0)
ENDROUTINE

%-----
% Release both input and output parts
%-----
ROUTINE RELEASE_HDLC(INTEGER : INLDN, OUTLDN)
    INTEGER : IOFLAG

    0 =: IOFLAG
    Monitor_Call('ReleaseResource', INLDN, IOFLAG)

    1 =: IOFLAG
    Monitor_Call('ReleaseResource', OUTLDN, IOFLAG)
ENDROUTINE

%-----
% Initialize the HDLC device
% Sends DEVINI command via DCB
% Returns 0 on success, error code otherwise
%-----
INTEGER ROUTINE INIT_HDLC(INTEGER : OUTLDN)
    INTEGER : FUNC, USIZE, MSIZE, STATUS

    4 =: TXDCB(0)           % Command = DEVINI
    0 =: TXDCB(1)           % Clear status
    0 =: TXDCB(2)           % Clear status

    0 =: FUNC                % 0 = send DCB to driver
    6 =: USIZE               % 3 words * 2 = 6 bytes
    2048 =: MSIZE            % Max DCB size in bytes

    Monitor_Call('HDLCfunction', FUNC, OUTLDN, TXDCB, USIZE, MSIZE, STATUS)

    IF STATUS <> 0 THEN
        CALL PRINTMSG('DEVINI failed, error: ')

```

```

        CALL PRINTNUM(STATUS)
        CALL NEWLINE
        RETURN (STATUS)
FI

CALL PRINTMSG('HDLC device initialized')
CALL NEWLINE

RETURN (0)
ENDROUTINE

%-----
% Send a data buffer over HDLC
% DATA      = array of words to send
% NWORDS     = number of data words
% OUTLDN     = output logical device number
% Returns 0 on success, error code otherwise
%-----
INTEGER ROUTINE SEND_DATA(INTEGER ARRAY : DATA, INTEGER : NWORDS, OUTLDN)
    INTEGER : FUNC, USIZE, MSIZE, STATUS, I

    % Build TRANS DCB
    1 =: TXDCB(0)           % Command = TRANS
    0 =: TXDCB(1)           % Clear status
    0 =: TXDCB(2)           % Clear status

    % Copy payload into DCB starting at word 3
    FOR I := 0 TO NWORDS-1 DO
        DATA(I) =: TXDCB(I + 3)
    OD

    0 =: FUNC                % 0 = send DCB to driver
    (NWORDS + 3) * 2 =: USIZE % Total DCB size in bytes
    2048 =: MSIZE            % Max DCB size

    CALL PRINTMSG('Sending ')
    CALL PRINTNUM(NWORDS)
    CALL PRINTMSG(' words...')
    CALL NEWLINE

    Monitor_Call('HDLCfunction', FUNC, OUTLDN, TXDCB, USIZE, MSIZE, STATUS)

```

```

IF STATUS <> 0 THEN
    CALL PRINTMSG('SEND error, status: ')
    CALL PRINTNUM(STATUS)
    CALL PRINTMSG(' DCB status1: ')
    CALL PRINTOCT(TXDCB(1))
    CALL PRINTMSG(' status2: ')
    CALL PRINTOCT(TXDCB(2))
    CALL NEWLINE
    RETURN (STATUS)
FI

CALL PRINTMSG('Send OK, driver status: ')
CALL PRINTOCT(TXDCB(1))
CALL NEWLINE

RETURN (0)
ENDROUTINE

%-----
% Receive a data buffer from HDLC
% DATA      = array to receive into
% MAXW       = max words to receive
% NWORDS     = actual words received (output)
% INLDN      = input logical device number
% DOWAIT     = 1 to block, 0 to poll
% Returns 0 on success, 3 if no data (poll mode)
%-----
INTEGER ROUTINE RECV_DATA(INTEGER ARRAY : DATA,
                          INTEGER : MAXW, NWORDS, INLDN, DOWAIT)
    INTEGER : FUNC, USIZE, MSIZE, STATUS, I, DATAWORDS

    % Clear receive DCB
    FOR I := 0 TO MAXW + 2 DO
        0 =: RXDCB(I)
    OD

    1 =: FUNC
    (MAXW + 3) * 2 =: USIZE
    DOWAIT =: MSIZE

    % 1 = receive DCB from driver
    % Buffer capacity in bytes
    % 1 = wait, 0 = poll

    Monitor_Call('HDLCfunction', FUNC, INLDN, RXDCB, USIZE, MSIZE, STATUS)

```

```

IF STATUS <> 0 THEN
  IF STATUS = 3 THEN
    % No DCB in queue -- normal for poll mode
    0 =: NWORDS
    RETURN (3)
  FI
  CALL PRINTMSG('RECV error, status: ')
  CALL PRINTNUM(STATUS)
  CALL NEWLINE
  0 =: NWORDS
  RETURN (STATUS)
FI

```

```

% Check that it is a TRANS DCB (data frame)
IF RXDCB(0) <> 1 THEN
  CALL PRINTMSG('Non-data DCB received, command: ')
  CALL PRINTNUM(RXDCB(0))
  CALL NEWLINE
  0 =: NWORDS
  RETURN (-1)
FI

```

```

% Calculate data words received
% USize now holds actual received size in bytes
USIZE / 2 - 3 =: DATAWORDS
IF DATAWORDS < 0 THEN
  0 =: DATAWORDS
FI
IF DATAWORDS > MAXW THEN
  MAXW =: DATAWORDS
FI

```

```

DATAWORDS =: NWORDS

```

```

% Copy payload out of DCB
FOR I := 0 TO DATAWORDS-1 DO
  RXDCB(I + 3) =: DATA(I)
OD

```

```

RETURN (0)
ENDROUTINE

```

```

%-----
% Print a buffer as octal and decimal values
%-----
ROUTINE DUMP_BUFFER(INTEGER ARRAY : DATA, INTEGER : NWORDS)
    INTEGER : I

    CALL PRINTMSG('Buffer dump (')
    CALL PRINTNUM(NWORDS)
    CALL PRINTMSG(' words):')
    CALL NEWLINE

    FOR I := 0 TO NWORDS-1 DO
        CALL PRINTMSG('  [')
        CALL PRINTNUM(I)
        CALL PRINTMSG('] = ')
        CALL PRINTOCT(DATA(I))
        CALL PRINTMSG(' (')
        CALL PRINTNUM(DATA(I))
        CALL PRINTMSG(')')
        CALL NEWLINE
    OD
ENDROUTINE

```

```

%-----
% MAIN PROGRAM
%-----
PROGRAM : MAIN

    INTEGER : INLDN, OUTLDN, STATUS, NRECV, I

    INISTACK STACK

    CALL PRINTMSG('=== HDLC Raw Send/Receive ===')
    CALL NEWLINE

    %
    % SET THESE TO YOUR HDLC DEVICE LDNs
    % Find them with @STATUS-DEVICE or SINTRAN-SERVICE-PROGRAM
    % Input and output have separate LDNs
    %
    560 =: INLDN                % HDLC input LDN  (CHANGE THIS)
    561 =: OUTLDN               % HDLC output LDN (CHANGE THIS)

```

```

CALL PRINTMSG('Input LDN: ')
CALL PRINTNUM(INLDN)
CALL NEWLINE
CALL PRINTMSG('Output LDN: ')
CALL PRINTNUM(OUTLDN)
CALL NEWLINE

% ---- STEP 1: Reserve the device ----
CALL PRINTMSG('Reserving HDLC device...')
CALL NEWLINE
CALL RESERVE_HDLC(INLDN, OUTLDN) =: STATUS
IF STATUS <> 0 THEN
    CALL PRINTMSG('Reserve FAILED')
    CALL NEWLINE
    CALL EXIT
FI
CALL PRINTMSG('Device reserved')
CALL NEWLINE

% ---- STEP 2: Initialize the HDLC link ----
CALL INIT_HDLC(OUTLDN) =: STATUS
IF STATUS <> 0 THEN
    CALL PRINTMSG('Init FAILED, releasing device')
    CALL NEWLINE
    CALL RELEASE_HDLC(INLDN, OUTLDN)
    CALL EXIT
FI

% ---- STEP 3: Fill test payload ----
%
% Your custom protocol header + data goes here
% This example uses a simple format:
%   Word 0: Message type
%   Word 1: Sequence number
%   Word 2: Payload length (in words)
%   Word 3+: Application data
%
1 =: TESTDATA(0)           % Message type = 1 (data)
0 =: TESTDATA(1)           % Sequence number = 0
8 =: TESTDATA(2)           % Payload = 8 words
101B =: TESTDATA(3)        % 'A'
102B =: TESTDATA(4)        % 'B'

```

```
103B =: TESTDATA(5)           % 'C'
104B =: TESTDATA(6)           % 'D'
105B =: TESTDATA(7)           % 'E'
106B =: TESTDATA(8)           % 'F'
107B =: TESTDATA(9)           % 'G'
110B =: TESTDATA(10)          % 'H'
```

```
CALL PRINTMSG('--- TX Data ---')
CALL NEWLINE
CALL DUMP_BUFFER(TESTDATA, 11)
```

```
% ---- STEP 4: Send the data ----
CALL SEND_DATA(TESTDATA, 11, OUTLDN) =: STATUS
IF STATUS <> 0 THEN
    CALL PRINTMSG('Transmit error')
    CALL NEWLINE
FI
```

```
% ---- STEP 5: Wait for a response (blocking) ----
CALL PRINTMSG('--- Waiting for RX data ---')
CALL NEWLINE
```

```
CALL RECV_DATA(RXDATA, 256, NRECV, INLDN, 1) =: STATUS
```

```
IF STATUS = 0 THEN
    CALL PRINTMSG('--- RX Data ---')
    CALL NEWLINE
    CALL DUMP_BUFFER(RXDATA, NRECV)
ELSE
    CALL PRINTMSG('No data received, status: ')
    CALL PRINTNUM(STATUS)
    CALL NEWLINE
FI
```

```
% ---- STEP 6: Cleanup ----
CALL PRINTMSG('Releasing device...')
CALL NEWLINE
CALL RELEASE_HDLC(INLDN, OUTLDN)
```

```
CALL PRINTMSG('Done.')
CALL NEWLINE
```

```
ENDROUTINE
```


ENDMODULE

Compilation and Execution

```
@PLANC HDLC-RAW,,HDLC-RAW
@NRL HDLC-RAW,,HDLC-RAW
@HDLC-RAW
```

Expected Terminal Output (successful run)

```
=== HDLC Raw Send/Receive ===
Input LDN: 560
Output LDN: 561
Reserving HDLC device...
Device reserved
HDLC device initialized
--- TX Data ---
Buffer dump (11 words):
  [0] = 1 (1)
  [1] = 0 (0)
  [2] = 10 (8)
  [3] = 101 (65)
  [4] = 102 (66)
  [5] = 103 (67)
  [6] = 104 (68)
  [7] = 105 (69)
  [8] = 106 (70)
  [9] = 107 (71)
  [10] = 110 (72)
Sending 11 words...
Send OK, driver status: 0
--- Waiting for RX data ---
--- RX Data ---
Buffer dump (N words):
  [0] = ...
  ...
Done.
```

11. Debugging Checklist

Before first transmission

- ☐ Correct LDNs obtained from @STATUS-DEVICE
- ☐ Input LDN reserved with IOFlag=0
- ☐ Output LDN reserved with IOFlag=1
- ☐ DEVINI DCB sent successfully (Status=0)
- ☐ Remote end is connected and initialized

Transmission problems

Symptom	Check	Fix
Status = 1	LDN not reserved	Call MON 122B before MON 201B
Status = 2	Wrong LDN	Verify with @STATUS-DEVICE
Status = 5	USize is wrong	Recalculate: $(words + 3) * 2$
Status = 6	Using input LDN for send	Swap to output LDN
Status = 7	MSize too small	Set MSize $\geq$ USize
DCB Status1 bit 1 set (TXUND)	DMA underrun	Retry; reduce system load
DCB Status1 bit 15 set (SILFO)	Illegal format	Check DCB structure; reinitialize

Reception problems

Symptom	Check	Fix
Status = 3 (polling)	No data in queue	Normal -- retry later
Status = 6	Using output LDN for receive	Swap to input LDN

Symptom	Check	Fix
Hangs on blocking receive	No remote transmitter	Use poll mode (MSize=0) with timeout logic
DCB command not 1	Non-data frame received	Handle RESET/DEVCL/DEVSTA or ignore
DCB Status1 bit 11 set (EMTY)	Receiver buffer exhaustion	FATAL -- reinitialize device
DCB Status1 bits 13-14 set	X.21 protocol error	Check physical link, X.21 interface

Appendix A: LDN Discovery

HDLC logical device numbers are assigned during system generation and can be found using SINTRAN commands.

Method 1: @STATUS-DEVICE

```
@STATUS-DEVICE
```

Lists all devices with their LDNs, types, and current reservation status.
HDLC devices are listed with type indicators for input and output parts.

Method 2: SINTRAN-SERVICE-PROGRAM

```
@SINTRAN-SERVICE-PROGRAM
* LIST-DEVICE
```

Shows the complete device table including HDLC controllers and their assigned LDN pairs.

Typical LDN Assignment

Device	Input LDN	Output LDN	Notes
HDLC channel 0	560	561	First HDLC link
HDLC channel 1	562	563	Second HDLC link
HDLC channel 2	564	565	Third HDLC link

Important: These are example values. Actual LDNs depend on system configuration and may differ on your installation. Always verify with @STATUS-DEVICE .

Appendix B: Advanced -- Direct Register Access

For expert users who need to bypass the HDLC driver entirely and control the COM5025 hardware directly via MON 31B (IOInstruction).

Reading Receiver Status

```
INTEGER : RegContents, DevRegAddr, ContentsAfter
```

```
% Read RRTS (receiver transfer status)
```

```
% DevRegAddr = HDEV base + 10 (offset for RRTS)
```

```
0 =: RegContents
```

```
HDEV + 10B =: DevRegAddr
```

```
Monitor_Call('IOInstruction', RegContents, DevRegAddr, ContentsAfter)
```

```
% ContentsAfter now holds RRTS bit pattern
```

Reading Transmitter Status

```
% Read RTTS (transmitter transfer status)
0 =: RegContents
HDEV + 12B =: DevRegAddr
Monitor_Call('IOInstruction', RegContents, DevRegAddr, ContentsAfter)

% Test for success: (ContentsAfter AND 100002B) should be 0
```

Emergency Stop

```
% Disable receiver
0 =: RegContents
HDEV + 11B =: DevRegAddr           % WRTC
Monitor_Call('IOInstruction', RegContents, DevRegAddr, ContentsAfter)

% Disable transmitter
0 =: RegContents
HDEV + 13B =: DevRegAddr           % WTTC
Monitor_Call('IOInstruction', RegContents, DevRegAddr, ContentsAfter)
```

Warning: Direct register access requires knowing the HDEV base address for your specific HDLC controller. This address is system-dependent and must be found via SINTRAN-SERVICE-PROGRAM or the device driver source code. Incorrect writes to hardware registers can crash the controller or the entire system.

Source References

Document	ND Number	Section	Content
SINTRAN III Monitor Calls	ND-860228.2 EN	Pages 306-307	HDLCfunction specification
SINTRAN III Monitor Calls	ND-860228.2 EN	Page 419	ReserveResource specification

Document	ND Number	Section	Content
SINTRAN III Monitor Calls	ND-860228.2 EN	Page 411	ReleaseResource specification
SINTRAN III Monitor Calls	ND-860228.2 EN	Page 141	DeviceControl specification
SINTRAN III Monitor Calls	ND-860228.2 EN	Page 327	IOInstruction specification
PLANC Reference Manual	ND-60.117.5 EN	--	PLANC language specification
HDLC Driver Source	MP-P2-HDLC-DRIV.NPL	--	SINTRAN HDLC driver (626 lines)
COM5025 Datasheet	SMC/AMD	--	Multi-Protocol Communications Controller